A Token-Budget Limiter for Destructive System Calls in xv6-riscv

Baviskar Darpan Chandrashekhar, Shrey Shah, Luv Sharma
Department of Computer Science and Engineering
Indian Institute of Technology Jodhpur, India
{b24cs1020, b24cs1111, b24cs1121}@iitj.ac.in

***Abstract*—We modify xv6-riscv so that each process group holds a token budget that is spent by a small set of destructive system calls (`unlink`, `write`, `exec`). Budgets refill lazily with time. When a group runs out, the kernel either blocks the caller until tokens accrue or fails the call with a dedicated error code, depending on the group's mode. Control over budgets is restricted by a capability that is inherited across `fork` and `exec` and can only be given up, never acquired. We check the implementation with an in-guest test suite (110 assertions), a workload driver graded by an independent host-side oracle (28 scenarios, 959 checks), mutation testing of the oracle (12 of 12 mutants caught), and the unmodified xv6 `usertests` (65 of 65 pass with the limiter active).**

***Index Terms*—xv6, operating systems, system calls, token bucket, resource limiting, capabilities**

I. Introduction

Classic Unix gives a process very little protection against its own mistakes: a runaway loop that calls `unlink` or overwrites files will do so at full speed. This lab adds a simple rate-limiting policy to xv6-riscv [1]. Calls that can destroy data are charged against a per-group token bucket, so a confined group can do only a bounded amount of damage per unit time.

The design goals were: (i) the charge happens before the call runs, so a denied call has no side effects; (ii) a denial is distinguishable from ordinary errors; (iii) a confined process cannot raise its own limit; and (iv) the behaviour can be checked against the lab specification without trusting the kernel's own bookkeeping.

II. System Design

A. Overview

Fig. 1 shows the components. The system call dispatcher computes a cost for each call and asks the budget engine to charge the caller's group. The engine keeps a fixed table of 20 groups. The file system records the creating group's id in each inode, which the cost function uses to tell "own" files from "other" files.

B. Cost Model

Costs are per system call, not per byte. Because the xv6 `printf` issues one `write` per character, console output is expensive relative to its length. Table I lists the prices.

TABLE I
Token Cost per System Call

Call	Condition	Cost
`unlink`	always	10
`write`	console (fd 1, 2)	1
`write`	file of another group	3
`write`	file of own group	0
`write`	pipe	0
`exec`	binary of another group	5
`exec`	binary of own group	0
others	—	0

C. Charge Path

The dispatcher charges the group before the system call body runs:

```
int cost   = syscall_cost(num, p);
int denial = charge(p->gid, cost);
if (denial < 0) {
  p->trapframe->a0 = denial;
  return;   /* call never runs */
}
p->trapframe->a0 = syscalls[num]();
```

A denied call therefore has no side effects. Calls with zero cost, and unknown system call numbers, are never charged. Denial returns `BUDGET_DENIED` (-42) instead of -1, so a caller can tell a budget refusal apart from, for example, `ENOENT`.

D. Blocking and Failing

Each group runs in one of two modes. In FAIL mode, `charge()` returns `BUDGET_DENIED` as soon as the bucket cannot cover the cost. In BLOCK mode the caller sleeps on `ticks` and re-checks after each timer tick, since refill is computed lazily from elapsed time.

```
charge(gid, cost):
  if cost <= 0: return 0
  lock; refill up to now
  if BLOCK and budget < cost and
     (max < cost or refill <= 0):
    n_refused++; return DENIED
  while budget < cost:
    if FAIL: n_refused++; return DENIED
    n_blocked++; sleep(&ticks); refill
  budget -= cost; n_allowed++
  return 0
```

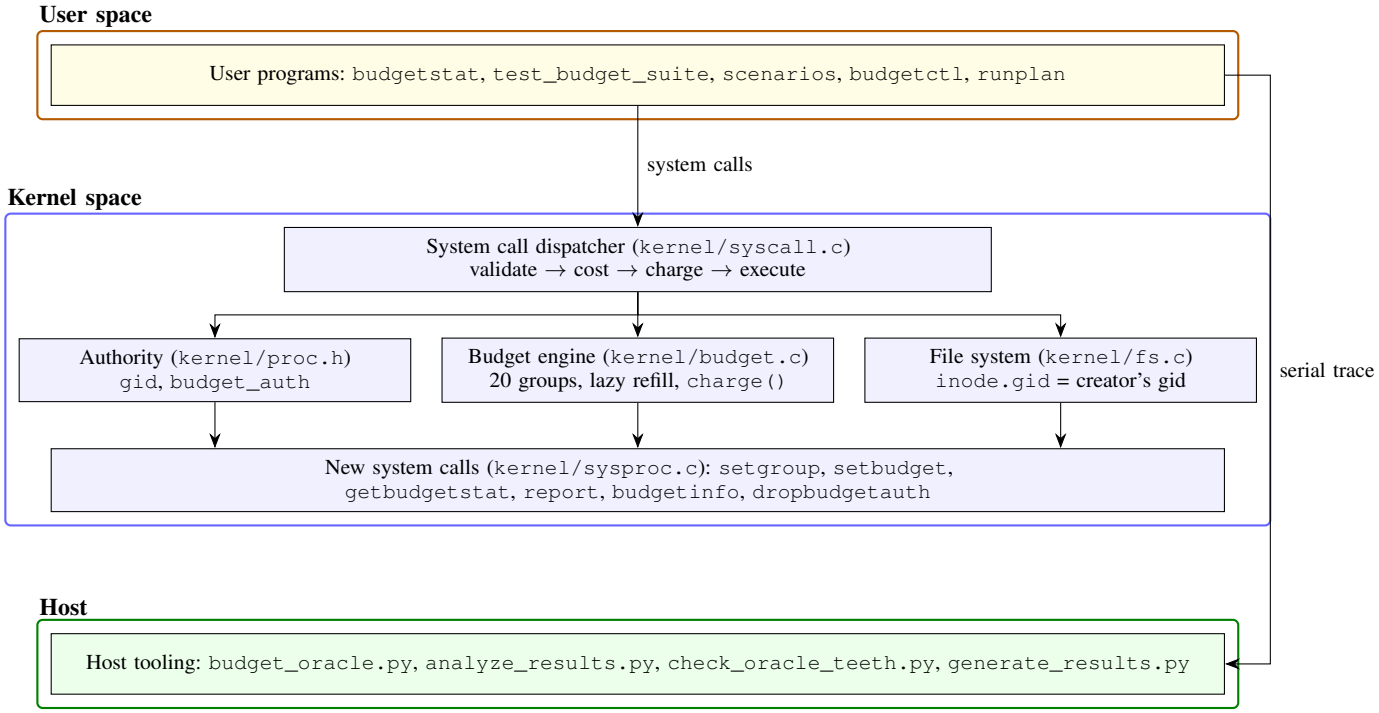


Fig. 1. Architecture. User programs enter the kernel through the dispatcher, which charges the caller's group before running the call. Serial output from the guest is graded offline by the host oracle.

The early check in BLOCK mode handles prices that can never be paid, either because the bucket ceiling is below the cost or because the refill rate is zero. Without it the caller would sleep forever.

E. Authority Model

Changing a group's budget or moving a process to another group requires the `budget_auth` flag in `struct proc`. The flag follows a simple lifecycle:

- `userinit()` creates the first process with `gid` 0 and `budget_auth` set. No other process starts with it.
- `fork` and `exec` preserve both fields.
- `freeproc()` clears the flag so a recycled slot starts clean.
- `dropbudgetauth()` clears it and returns 1 if it was set and 0 if it was already gone.
- `setgroup` and `setbudget` check the flag first and return `BUDGET_NOAUTH` (-43) without it.

The flag is inherited but never acquired, so a process that has dropped it cannot regain it. Read-only calls (`getbudgetstat`, `report`, `budgetinfo`) remain available to everyone.

F. Unmetered Reporting

Console `write` must cost one token, which means a test that exhausts its own budget can no longer print its result. We therefore added `report(buf, len)`, which prints through the kernel's `printk` at no charge and is counted separately in `n_reported` and `report_bytes`. The user-level wrapper `rprintf()` formats once with `snprintf` and sends the buffer through `report`. The usual `printf` still goes through the charged `write`, so the lab requirement is unchanged.

III. Implementation Issues

Six problems took most of the debugging time.

A. Silenced Test Output

Verdicts were lost when the test's own `printf` drained the budget. This motivated the free reporting path above.

B. Ambiguous Error Code

Returning -1 for a denial made it look like `ENOENT`. The kernel now publishes `BUDGET_DENIED` and `BUDGET_NOAUTH` through `budgetinfo()`, and the tests assert against those values.

C. BLOCK Mode Hang

With a ceiling of 9 and a price of 10 a blocked caller never woke. The early check in `charge()` fixes this.

D. Stale Inode Group

`namei()` returns an unlocked inode, so reading its group id in the `exec` charge path could see stale data. The read is now wrapped in `ilock()` and `iunlockput()`.

E. Lost Refill on Policy Change

`setbudget()` reset `last_refill` without crediting the tokens earned so far. A new `budget_bank()` credits them first; it is called from `refill()`, `sys_setbudget()` and the snapshot path, and the accrual clock only advances after banking.

TABLE II
VERIFICATION SUMMARY

Component	Checks	Passed	Time (s)
Test suite	110	110	∼158
Scenarios	959	959	∼173
Mutants caught	12	12	—
`usertests`	65	65	∼152

Wall-clock time under QEMU, single CPU.

F. Self-Service Budget Increases

Originally any process could call `setbudget` on its own group and escape its limit. The authority flag described above closes this.

IV. VERIFICATION

A. Method

Test suite. `test_budget_suite` runs inside xv6 and makes 110 assertions. Expected values are computed from counter deltas rather than hard-coded, and the price table is read from the kernel through `budgetinfo()` so that tests and kernel cannot drift apart.

Scenarios and oracle. The `scenarios` program runs 28 workloads and prints a trace but asserts nothing. A separate host script, `budget_oracle.py`, was written from the lab specification rather than from the kernel source. It parses the trace, derives the expected decision, price, counters and bucket level for each operation, and compares them with what the kernel reported. The oracle applies six groups of checks: (1) the published price table against the lab specification; (2) per-operation decisions, prices, counters and bucket; (3) fan-out aggregates, i.e. $\lfloor \text{bucket}/\text{price} \rfloor$ allowed calls; (4) rejection of invalid inputs; (5) confinement, covering seven escape attempts that must be blocked with the policy unchanged and the workload still charged; and (6) cross-run invariants, such as free classes never moving counters and denials never charging.

Mutation testing. To show the oracle can fail, we edited the trace in 12 ways, each changing a single field, and confirmed that a specific check rejects each one.

Upstream tests. The stock xv6 `usertests` are run with the limiter enabled, to confirm that ordinary programs are unaffected.

B. Results

Table II summarises the runs. There were no failures. In the `usertests` run the limiter accounted for 55,012 calls and throttled none.

V. DISCUSSION

A. Per-Call Pricing

Charging per call rather than per byte keeps the kernel simple and matches the lab specification, but it treats a one-byte and a one-page `write` identically. Combined with the one-`write`-per-character `printf`, console output is the largest consumer of tokens in practice.

B. Trusting the Oracle

The oracle is only useful if it can say no. The mutation runs are a minimal check of that; they do not prove the oracle is complete, since each mutant alters a single field.

C. Remaining Work

The enforcement engine is complete. A console front-end and live graphing of bucket levels are not yet implemented.

VI. CONCLUSION

We added a per-group token budget for `unlink`, `write` and `exec` to xv6-riscv, with FAIL and BLOCK behaviour, lazy refill, and an inherited, drop-only authority flag. On QEMU the implementation passes its own test suite, the independently written oracle, and the upstream `usertests` unchanged.

REFERENCES

[1] R. Cox, M. F. Kaashoek, and R. Morris, *xv6: A Simple, Unix-like Teaching Operating System*, MIT, 2022.